UNIVERSITY OF MANNHEIM
School of Business Informatics and Mathematics

Test Suite Minimisation in LASSO Using Stimulus-Response Matrices

Bachelor Thesis

submitted: 01.December 2025

by: Laith Philipp Sandouk
laith.sandouk@students.uni-mannheim.de
born May 15th, 2005
in Mannheim

Student ID Number: 1993811

Chair of Software Engineering
School of Business Informatics and Mathematics
University of Mannheim
D – 68159 Mannheim
Phone: +49 621-181-3912, Fax +49 621-181-3909
Internet: http://swt.informatik.uni-mannheim.de

Abstract

Large-scale industrial software systems face significant challenges in executing test suites efficiently. In a commercial SAP environment, Bach et al. report test suites comprising more than 130,000 tests requiring hundreds of hours of sequential runtime [2]. Such execution times render continuous testing computationally infeasible and economically prohibitive. The study further revealed extensive redundancy within these test suites, implying that substantial execution cost yields only marginal additional coverage benefit. However, this is not an isolated case, test-suite redundancy is a widespread and well-documented issue in industrial software development. While this computational burden cannot be eliminated entirely, its impact can be substantially reduced through test-suite minimisation, which systematically removes redundant tests with respect to defined adequacy criteria such as statement coverage, branch coverage, or mutation score [19]. However, naive reduction approaches risk compromising fault-detection effectiveness, as tests with unique defect-revealing capabilities may be inadvertently excluded.

This thesis investigates test-suite minimisation in the context of the *Large-Scale Software Observatorium* (LASSO) [11], a research platform for automated selection, execution, and comparative analysis of software systems. LASSO integrates open-source repositories and combines static and dynamic analyses within a unified framework designed for language independence. Its current implementation focuses on Java and Python systems and enables empirically grounded studies of *Big Code* ecosystems (large, diverse corpora of source code enriched with metadata such as bug-fix histories, version control traces, and execution outcomes [1]).

Our approach leverages *Stimulus–Response Matrices* (SRMs) [11], a two-dimensional representation mapping test stimuli to observed execution outcomes. We augment SRM entries with per-test coverage and mutation information, enabling minimisation algorithms to operate on fine-grained behavioural data rather than language-specific code structures.

Through a targeted literature survey encompassing greedy heuristics, exact optimisation, search-based, and data-driven minimisation strategies, we comparatively assessed candidate algorithms along seven established criteria: SRM compatibility, coverage retention, reduction effectiveness, fault-detection retention, mutation-coverage preservation, computational scalability, and implementation feasibility. The Mutation-Aware Enhanced HGS (MA-EHGS) algorithm emerged as the most promising candidate. MA-EHGS extends the empirically validated Enhanced HGS baseline [6] by integrating mutation weighting to balance coverage preservation and fault-detection power [8].

We successfully integrated MA-EHGS into LASSO and validated the implementation on real-world SRMs. Our evaluation achieved reductions within the ranges reported in prior studies, while maintaining high coverage and mutation-scores [6, 8]. The algorithm executes in $O(nm)$ time, ensuring computational feasibility for large-scale software analysis workflows.

Contents

List of Figures

List of Tables

List of Abbreviations

CFG	Control Flow Graph
CI	Continuous Integration
EHGS	Enhanced HGS
GA	Genetic Algorithm
HGS	Harrold–Gupta–Soffa algorithm
ILP	Integer Linear Programming
LASSO	Large-Scale Software Observatorium
LSL	LASSO Scripting Language
MA-EHGS	Mutation-Aware Enhanced HGS
SRM	Stimulus-Response Matrix
TSM	Test Suite Minimisation

1. Introduction

Software testing ensures that program modifications do not introduce defects. As systems evolve, test suites expand, increasing execution cost and maintenance effort. Suites often combine human-written, automatically generated, and, increasingly, AI-generated tests. Tools such as Randoop [16] and EvoSuite [5] can generate thousands of cases, and in large projects, execution may require days or weeks [17]. While comprehensive suites achieve high coverage and mutation scores, they frequently contain redundant tests whose requirements are subsumed by others, causing prolonged execution, higher costs, and elevated maintenance overhead.

Test-suite minimisation systematically removes redundant tests while preserving fault-detection capability [19]. Techniques can be broadly divided into *coverage-preserving* approaches, which guarantee that all coverage requirements remain satisfied, and *coverage-relaxing* approaches, which permit limited coverage loss in exchange for stronger reduction. Minimisation can greatly decrease computational demand and feedback latency, but excessive reduction—especially in coverage-relaxing strategies—risks removing tests with unique defect-revealing potential. Effective minimisation therefore requires a careful balance between reduction efficiency and fault-detection retention.

1.1. Background and Motivation

Testing remains a major cost driver in software engineering. Studies estimate that verification and validation account for up to half of total development effort [3]. Although these figures originate from traditional development models, the cost pressure persists in modern continuous integration, where automated or AI-generated suites may contain tens of thousands of tests [17, 2]. Executing all tests after each modification is often infeasible, motivating scalable approaches that reduce suite size without compromising quality. Test-suite minimisation addresses this challenge by identifying and removing redundant tests while retaining those essential for coverage and fault detection. As an optimisation problem, it is NP-hard [19] and therefore typically solved through heuristics or approximation algorithms, which are further analysed in Chapter 2.

1.2. LASSO Context

The *Large-Scale Software Observatorium* (LASSO) [11] is a research platform developed by the Chair of Software Engineering at the University of Mannheim. It provides an environment for the scalable analysis and observation of large software corpora, commonly referred to as *Big Code*. LASSO automates the collection and analysis of both static information (e.g., source code properties) and dynamic information (e.g., run-time execution behaviour) from extensive sets of executable open-source systems. Its domain-specific scripting language, the *LASSO Scripting Language* (LSL), enables researchers to define reproducible pipelines for tasks such as code search, automated unit-test generation, and large-scale *Test-Driven Software Experiments* (TDSEs). These capabilities make LASSO a central infrastructure for empirical software engineering and for evaluating emerging technologies, including Generative AI for code.

LASSO employs *Stimulus–Response Matrices* (SRMs) [11] as a unified representation for systematic comparison. An SRM is a two-dimensional matrix in which rows denote test stimuli, columns denote systems, and cells contain structured response objects that capture execution outcomes. From this representation, coverage metrics, mutation scores, and behavioural characteristics can be systematically extracted and analysed, enabling language-independent comparison of functionally equivalent implementations. Section 2.2 provides detailed SRM definitions and explains coverage and mutation data extraction.

While LASSO enables comprehensive behavioural and coverage analysis, it currently lacks an integrated mechanism for test-suite minimisation. This absence can lead to redundant test executions that accumulate significant computational overhead when scaled across the platform's extensive corpus. Incorporating minimisation within LASSO would help mitigate such redundancy while maintaining the behavioural coverage required for reliable system analysis.

1.3. Research Objectives and Questions

This thesis investigates the integration of test-suite minimisation within the LASSO framework. Specifically, it aims to design, evaluate, and implement an SRM-based minimisation approach that maintains language independence and scalability. The following research questions guide the study:

1. **RQ1:** Which test-suite minimisation techniques demonstrate the highest effectiveness and practical applicability according to current literature?
2. **RQ2:** What evaluation criteria are most appropriate for assessing minimisation techniques within LASSO's SRM-based environment?
3. **RQ3:** Which algorithmic approach best balances reduction effectiveness, coverage preservation, and computational scalability for integration with LASSO?
4. **RQ4:** How can the selected approach be integrated into LASSO's analysis pipeline while maintaining language independence?

1.4. Implementation Strategy

This thesis implements minimisation directly on LASSO's SRM abstraction rather than integrating external tools. Existing frameworks (OpenClover, PIT) support limited languages and require source code access or language-specific instrumentation, conflicting with LASSO's language-independent design.

SRMs already encapsulate all information necessary for behavioural minimisation: test identifiers, execution results, coverage metrics, and mutation data. Operating exclusively on this structure maintains language independence, ensures scalability by processing compact SRM representations instead of source artefacts, and promotes architectural cohesion through seamless integration with LASSO's analysis pipeline.

1.5. Thesis Structure

The remainder of this thesis is organised as follows:

Chapter 2 introduces coverage, mutation testing, SRMs, and formally defines the minimisation problem.

Chapter 3 reviews related work across greedy, exact, search-based, and data-driven techniques.

Chapter 4 defines evaluation criteria (C1–C7) and presents the scoring framework for algorithm comparison under SRM constraints.

Chapter 5 applies the framework to rank algorithms and justify the selection of the Mutation-Aware Enhanced HGS approach.

Chapter 6 details the implementation and integration into LASSO.

Chapter 7 discusses limitations, trade-offs, and deployment considerations.

Chapter 8 concludes and outlines future work.

2. Fundamentals

This chapter establishes the theoretical foundation for this work. It introduces the core concepts of test coverage and mutation testing, explains the *Stimulus–Response Matrix* (SRM) representation used within LASSO, and formalises the test-suite minimisation problem, including trade-offs between coverage preservation and reduction strength.

2.1. Test Coverage and Mutation Score

Test coverage quantifies the proportion of program elements exercised by a test suite [20]. Coverage metrics indicate structural thoroughness but provide limited insight into fault-detection capability.

2.1.1. Coverage Metrics

Coverage criteria define program "parts" at different granularity levels [20]. Common coverage types include:

- **Instruction Coverage:** proportion of bytecode instructions executed.
- **Line Coverage:** proportion of source lines executed.
- **Branch Coverage:** proportion of conditional branches whose true/false outcomes are both executed.
- **Method Coverage:** proportion of methods invoked at least once.
- **Complexity Coverage:** proportion of independent execution paths exercised (cyclomatic complexity).
- **Class Coverage:** proportion of classes instantiated or whose static methods execute.

Multi-level coverage captures execution at different granularities. High line coverage with low branch coverage may miss conditional faults, while high method coverage without path coverage may miss defects in complex methods. Combining several metrics provides a more complete assessment of test quality.

2.1.2. Mutation Score

Mutation testing measures test effectiveness by introducing small syntactic changes (*mutants*) into the program and observing whether tests detect the injected faults [9]. The *mutation score* is defined as:

$$\text{Mutation Score} = \frac{\text{Number of mutants killed}}{\text{Total number of mutants}} \times 100\,\%.$$

A mutant is *killed* if the test causes observable behavioural divergence from the original program.

Mutation Types. Two principal mutation styles are distinguished [15]:

1. **Weak Mutation:** detects state deviations immediately after a mutated statement executes.
2. **Strong Mutation:** detects externally visible behavioural differences at program output.

Weak mutation captures local sensitivity to code changes; strong mutation quantifies global behavioural divergence. Mutation scores thereby approximate a suite's fault-detection capability.

2.2. Stimulus–Response Matrices

Stimulus–Response Matrices (SRMs) constitute LASSO's central data structure for *Test-Driven Software Experimentation* [11]. They provide a unified, language-independent format for recording and comparing run-time behaviour under controlled stimuli.

2.2.1. Sequence and Actuation Sheets

Sequence Sheets describe ordered stimuli applied to systems under test. Each row represents one method invocation with inputs and expected outputs. **Actuation Sheets** record the corresponding observed outputs and other run-time data during execution.

Stimulus (Sequence Sheet)

Out	Operation	Service	Input
create	`create`	`'Stack`	
–	`push`	A1	42
42	`pop`	A1	

Observed Behaviour (Actuation Sheet)

Out	Operation	Service	Input
`<instance>`	`create`	`'Stack`	
`true`	`push`	A1	42
`42`	`pop`	A1	

Figure 2.1.: Relationship between a sequence sheet (stimulus) and its corresponding actuation sheet (observed behaviour).

2.2.2. From Sequence Sheets to SRMs

LASSO generalises stimulus-to-actuation mappings into multidimensional *Stimulus–Response Matrices* (SRMs):

$$M = [M_{ij}], \qquad M_{ij} = \text{Actuation}(t_i, v_j),$$

where each row t_i represents a stimulus (test), each column v_j a system variant, and each cell M_{ij} stores execution outcomes and associated analysis data such as coverage and mutation attributes.

	System 1	**System 2**	**System 3**	**System 4**
Test 1	pass	fail	pass	fail
Test 2	pass	pass	pass	pass
Test 3	pass	fail	fail	pass

Figure 2.2.: Example SRM fragment showing per-test outcomes across different system variants. Behavioural discrepancies can be used to compute mutation scores and coverage information.

SRMs unify functional results, coverage data, and mutation outcomes in a single matrix, enabling language-independent behavioural analysis.

2.3. The Test-Suite Minimisation Problem

2.3.1. Concept and Objectives

Test-suite minimisation seeks to remove redundant tests from an existing suite while maintaining sufficient coverage and fault-detection capability [19]. Given a suite $T = \{t_1, \ldots, t_n\}$ and a set of coverage requirements $R = \{r_1, \ldots, r_m\}$, each test t_i covers a subset $C(t_i) \subseteq R$. The objective is to find a smaller subset $S^* \subseteq T$ that provides comparable or identical coverage.

Two major formulations exist:

- **Coverage-preserving minimisation:** guarantees full coverage, $C(S^*) = C(T)$. It ensures no requirement is lost but often limits reduction potential.
- **Coverage-relaxing minimisation:** allows limited loss of coverage, $C(S^*) \subset C(T)$, to achieve greater reduction. This variant trades fault-detection certainty for smaller suite size.

Both forms address the same optimisation tension: *minimise suite size while retaining sufficient adequacy.* The general optimisation can be expressed as

$$S^* = \arg\min_{S \subseteq T} |S| \quad \text{subject to} \quad |C(S)| \geq \gamma \cdot |C(T)|,$$

where $0 < \gamma \leq 1$ controls the acceptable coverage retention threshold ($\gamma = 1$ for coverage-preserving, $\gamma < 1$ for coverage-relaxing).

2.3.2. Computational Complexity and Trade-offs

The coverage-preserving variant corresponds to the *minimum set-cover problem*, which is NP-complete [19]. No polynomial-time algorithm guarantees optimality for arbitrary inputs, and practical implementations rely on heuristics or approximations [7, 4]. Some greedy heuristics achieve linear complexity $O(nm)$ in practice, balancing scalability and quality.

Coverage-relaxing algorithms may yield stronger reductions but risk discarding unique defect-revealing tests, potentially lowering mutation scores or missing rare execution paths. Hence, every minimisation approach must balance the trade-off between computational efficiency, reduction rate, and fault-detection retention.

Original Test Suite

$T = \{t_1, \ldots, t_n\}$

$|T| = n$ tests

$C(T) = \bigcup_{i=1}^{n} C(t_i)$

Complete coverage

Reduced Test Suite

$S^* \subseteq T$

$|S^*| < n$ tests

$C(S^*) = C(T)$ (preserving)

or $C(S^*) \subset C(T)$ (relaxing)

Optimisation Goal

Minimise: $|S^*|$

Subject to: $C(S^*) \geq \gamma \cdot C(T)$, where $0 < \gamma \leq 1$

Complexity: NP-complete (set-cover equivalent)

Figure 2.3.: The test-suite minimisation problem formulated as an optimisation balancing suite size and coverage retention.

2.3.3. Minimisation within LASSO

In LASSO, each test corresponds to one SRM row. Coverage and mutation data stored in SRM cells provide the requirement mappings $C(t_i)$. Minimisation thus operates directly on SRMs by selecting a subset of rows S^* that preserves coverage and mutation adequacy across all variants:

$$C(S^*)_{\text{SRM}} \geq \gamma \cdot C(T)_{\text{SRM}}.$$

This SRM-based formulation supports both coverage-preserving and coverage-relaxing strategies while remaining language-independent, enabling scalable experimentation across diverse software systems.

2.3.4. Algorithmic Categories

Following established classifications [19, 12, 2], algorithms group into four categories: **greedy heuristics** (classical greedy, HGS [7]), **exact optimisation** (ILP, constraint-satisfaction [13]), **search-based methods** (genetic algorithms [19]), **data-driven methods** (clustering, overlap-aware [2]). Detailed explanation in Chapter 3.

3. Minimisation Algorithms

This chapter surveys test-suite minimisation algorithms across four categories: greedy heuristics, exact optimisation, search-based approaches, and data-driven techniques.

3.1. Greedy Heuristics

Greedy heuristics iteratively select tests covering the most uncovered requirements until complete coverage is achieved [19, 12], operating in polynomial time with predictable performance.

3.1.1. Classical Greedy

Classical greedy iteratively selects tests covering the most uncovered requirements [4, 19], achieving $O(nm)$ complexity with $O(\ln m)$ approximation ratio. Empirical studies show 60–93 % reduction (median 70 %) maintaining 100 % structural coverage, but mutation scores drop 3.5–21 % [14, 8].

3.1.2. Harrold–Gupta–Soffa

HGS prioritises requirements by rarity [7], operating in phases by cardinality with runtime complexity of $O(|T| \cdot \max(|T_i|))$ [19], where $|T|$ is the size of the original test suite and $\max(|T_i|)$ represents the cardinality of the largest group of test cases. HGS produces suites 0.37–2.92 % smaller than classical greedy with mutation differences (0.09–4.88 %) statistically insignificant ($p \gg 0.05$) [14].

3.1.3. Delayed Greedy

Delayed greedy [18] uses concept lattices to eliminate redundancies before greedy selection. It produces suites equal to or smaller than classical greedy in 75 % of cases [18, 19], achieving 65–74 % reduction while maintaining mutation scores [8]. Polynomial complexity, though exact bound unstated [18].

3.1.4. Enhanced HGS

Enhanced HGS [6] operates on a requirements matrix in two phases: (1) select tests uniquely covering any requirement, (2) iteratively select tests maximising uncovered requirements. Time complexity is $O(nm)$, matching classical greedy. Empirical validation on Siemens benchmarks demonstrated smaller suites than HGS and classical greedy while maintaining coverage [6].

3.1.5. Mutation-Aware Enhanced HGS

Mutation-Aware Enhanced HGS (MA-EHGS) extends the Enhanced HGS baseline by integrating mutation testing into the selection process. This algorithm represents the contribution of this thesis, addressing the documented coverage-mutation trade-off where coverage-only minimisation loses 3.5–21 % mutation score [14].

Requirements-Matrix Formulation

MA-EHGS operates on a requirements matrix $M \in \{0,1\}^{n \times m}$ where rows represent tests $T = \{t_1, \dots, t_n\}$ and columns represent requirements $R = \{r_1, \dots, r_m\}$. Entry $M_{ij} = 1$ indicates test t_i satisfies requirement r_j. Requirements comprise both structural coverage elements (lines, branches, methods from JaCoCo metrics) and mutation kills. Mutants are encoded as additional requirements, enabling unified treatment of structural and semantic properties.

Two-Phase Selection with Mutation Weighting

Phase 1: Essential Test Selection. The algorithm identifies requirements r_j covered by exactly one test ($|T_j| = 1$) and selects these essential tests into the minimised suite S^*. This applies to both structural requirements and mutation kills: if only one test kills a particular mutant, that test is essential.

Phase 2: Weighted Greedy Selection. After removing essential tests, the algorithm iteratively selects tests maximising a weighted score combining structural coverage and mutation kills:

$$\text{score}(t_i) = |\text{newCoverage}_i| + \beta \times |\text{newMutants}_i| \tag{3.1}$$

where $|\text{newCoverage}_i|$ counts uncovered structural requirements that t_i would satisfy, $|\text{newMutants}_i|$ counts unkilled mutants that t_i would kill, and $\beta \in [0, \infty)$ controls mutation emphasis. The default $\beta = 0.7$ provides coverage-dominant weighting where structural coverage remains primary but mutation coverage provides discriminatory power. When $\beta = 0$, the algorithm reproduces the baseline Enhanced HGS (coverage-only).

Complexity and Performance

MA-EHGS maintains $O(nm)$ time complexity and $O(nm)$ space complexity, matching classical greedy. The original HGS has runtime complexity $O(|T| \cdot \max(|T_i|))$ [19], where $|T|$ is the size of the test suite and $\max(|T_i|)$ represents the cardinality of the largest group of test cases. The mutation extension adds no asymptotic overhead: mutants are simply treated as additional requirements in the matrix. The empirically validated baseline [6] provides confidence in effectiveness, while the β-parameter permits tuning the coverage-mutation balance via grid search ($\beta \in \{0.5, 0.6, 0.7, 0.8, 0.9\}$). An unused parameter $\alpha = 0.3$ is reserved for future extensions (e.g., test execution cost weighting).

3.2. Exact Optimisation Approaches

Exact methods formulate minimisation as ILP or constraint-satisfaction problems [19, 12], guaranteeing optimality but exhibiting NP-hard exponential runtime.

3.2.1. ILP and REDUNET

ILP guarantees optimal solutions with $(m\Delta)^{O(m)}$ complexity [19]. REDUNET [13] achieves $> 3\times$ speedup via hybrid CFG-network optimisation with up to 90 %

reduction, but CFG reliance precludes SRM-only integration.

3.3. Search-Based and Metaheuristic Approaches

Search-based methods treat minimisation as multi-objective optimisation [19, 12], providing flexibility but requiring parameter tuning with higher overhead than greedy heuristics.

3.3.1. Genetic Algorithms

GAs evolve test subsets with $O(gnm)$ complexity, producing Pareto fronts trading coverage against suite size [19]. They require parameter tuning with longer runtime than greedy methods. No systematic mutation-aware GA evaluation has been reported.

3.4. Data-Driven and Similarity-Based Approaches

Data-driven methods utilise coverage properties to identify redundancies [2, 12], operating directly on coverage without requiring program structure.

3.4.1. Similarity-Based Methods

These compute distance metrics over coverage vectors with base complexity $O(n^2m)$. FAST-R achieves near-linear scalability via random projection and clustering [12]. No mutation evaluation has been reported.

3.4.2. Overlap-Aware Methods

Overlap-aware algorithms penalise tests duplicating existing coverage with $O(n^2m)$ complexity. Bach et al. achieved 50 % higher coverage within fixed time budgets versus classical greedy, completing under 10 seconds [2]. No mutation evaluation reported.

Table 3.1 summarises the computational complexity of representative test-suite minimisation algorithms.

Table 3.1.: Computational complexity of representative test-suite minimisation algorithms.

Algorithm	Time Complexity	Space Complexity	Reference
Classical Greedy	$O(nm)$	$O(nm)$	[4]
HGS	$O(\|T\| \cdot \max(\|T_i\|))^{\mathrm{d}}$	$O(nm)$	[19]
Delayed Greedy	$O(n^2m)$	$O(nm+n^2)$	[18]
Enhanced HGS	$O(nm)$	$O(nm)$	[6]
MA-EHGS	$O(nm)$	$O(nm)$	This work
ILP-Based	Exponential	$O(nm)$	[13]
REDUNET	Exponential*	$O(nm+\|V\|+\|E\|)^{\mathrm{a}}$	[13]
Genetic Algorithm	$O(gnm)^{\mathrm{b}}$	$O(pnm)^{\mathrm{c}}$	[19]
Similarity-Based	$O(n^2m)$	$O(nm+n^2)$	[12]
Overlap-Aware	$O(n^2m)$	$O(nm+n^2)$	[2]

[a] $|V|$ and $|E|$ denote control-flow graph vertices and edges.
[b] g denotes number of generations.
[c] p denotes population size.
[d] $|T|$ is test suite size, $\max(|T_i|)$ is largest group cardinality.

3.5. Summary

The surveyed literature reveals four principal findings informing LASSO algorithm selection:

Greedy variants exhibit minimal practical differences. Classical greedy, HGS, and delayed greedy produce statistically indistinguishable results ($p \gg 0.05$) [14], suggesting refinements provide limited benefit when overlap is moderate.

Coverage-mutation trade-off is substantial. Coverage-based minimisation consistently degrades fault-detection with losses of 3.5–21 % despite 100 % coverage [14, 17].

Scalability leaders employ diverse strategies. REDUNET achieves $> 3\times$ speedup but requires CFGs unavailable in SRM-only settings [13]. Overlap-aware methods show 74–77 % time savings [2]. Greedy variants maintain polynomial scalability.

Mutation analysis gap is widespread. Most algorithms lack mutation evaluation, limiting understanding of fault-detection characteristics and motivating mutation-enhanced approaches.

4. Evaluation Criteria and Scoring Framework

Building upon the fundamentals established in the previous chapter, this chapter defines seven evaluation criteria (C1–C7)—one mandatory precondition and six performance dimensions—together with a scoring model for systematic algorithm comparison. These criteria operationalise the research questions and provide the foundation for evaluating different minimisation algorithms in the next chapter.

4.1. Evaluation Criteria (C1–C7)

4.1.1. Mandatory Precondition

C1: SRM Compatibility (Mandatory) The algorithm must operate directly on *Stimulus–Response Matrix* (SRM) representations without requiring control-flow graphs, syntax trees, or language-specific structural analysis [11]. Algorithms failing C1 receive a score of zero and are excluded. This precondition guarantees language-independent execution across heterogeneous software systems.

4.1.2. Performance Criteria

C2: Coverage Retention Measures how much of the original structural coverage (instruction, line, branch) is retained after minimisation [20]. High retention indicates that the reduced suite maintains structural adequacy.

C3: Reduction Effectiveness Quantifies the proportion of test cases eliminated while preserving required coverage [19]. A high reduction rate reflects strong elimination of redundant tests and tangible computational savings.

C4: Fault Detection Retention Evaluates whether the minimised suite preserves the ability to reveal real software defects. Since redundancy with respect to coverage does not imply redundancy with respect to faults [10], this criterion measures the proportion of defect-revealing tests retained.

C5: Mutation-Coverage Preservation Assesses fault-detection capability via mutation testing [8, 14]. The mutation score—ratio of killed mutants to total mutants—acts as a fault-based adequacy metric complementing structural coverage.

C6: Computational Scalability Examines algorithm performance on large SRMs as the number of tests and requirements increases. Methods with near-linear complexity and efficient runtime behaviour are favoured [11, 2].

C7: Implementation Feasibility Considers practical integration factors such as implementation effort, parameter tuning, dependency overhead, and compatibility with LASSO's architecture [19]. Algorithms imposing heavy maintenance or configuration costs are penalised.

4.1.3. Justification of Criteria

The seven criteria align directly with the research questions from Section 1.3, ensuring coherence between theoretical goals and empirical assessment.

C1 (SRM Compatibility) is mandatory because LASSO operates entirely on SRM abstractions [11]. Language independence requires that algorithms process abstract requirement matrices rather than language-specific artefacts.

C2–C5 (Quality Preservation) address adequacy and fault detection. **C2** ensures structural completeness; **C3** quantifies the degree of reduction; **C4** safeguards against loss of fault-revealing tests as observed in Defects4J benchmarks [10]; and **C5** measures mutation-score retention, a strong empirical proxy for test effectiveness [8].

C6 (Computational Scalability) ensures the algorithm remains feasible for large-scale use cases where SRMs may contain thousands of rows and columns [11, 2].

C7 (Implementation Feasibility) recognises the cost of practical adoption. Even theoretically optimal algorithms may become impractical if they require complex configuration or structural changes to the LASSO pipeline [19].

4.2. Scoring Model

Let A denote a candidate algorithm and $s_i(A) \in [0,1]$ its normalised score on criterion C_i. Let $w_i \geq 0$ denote the relative weight for criterion C_i, with $\sum_{i=2}^{7} w_i = 1$. The total score is defined as:

$$\mathrm{Score}(A) = \begin{cases} 0, & \text{if } s_1(A) = 0 \text{ (fails C1)}, \\ \displaystyle\sum_{i=2}^{7} w_i \cdot s_i(A), & \text{otherwise.} \end{cases} \tag{4.1}$$

4.2.1. Rank-Based Point Assignment

Scores $s_i(A)$ for each criterion C_i are derived through rank-based evaluation, following standard regression-testing methodology [19]. For each performance criterion ($i \in \{2, \ldots, 7\}$), algorithms are ranked by performance; the top-ranked algorithm receives the highest score, and tied algorithms share the same rank. Subsequent ranks are offset accordingly (e.g., a tie for first shifts the next rank to third). This ensures equitable treatment of equivalent performers while maintaining meaningful differentiation across criteria.

5. Comparative Evaluation and Selection

This chapter evaluates algorithms against the criteria established in Chapter 4, presenting criterion-specific comparisons followed by multi-criteria scoring to select the optimal approach for LASSO integration.

5.1. Criterion-by-Criterion Evaluation

5.1.1. C1: SRM Compatibility

SRM compatibility determines whether an algorithm can operate using only stimulus-response matrices without requiring control-flow graphs, abstract syntax trees, or language-specific structural analysis.

Table 5.1.: C1: SRM Compatibility — algorithms requiring only coverage matrices.

Algorithm	**Compatible**	**Justification**
Classical Greedy	✓	Operates on coverage matrix only [4]
HGS	✓	Requires only requirement cardinalities [7]
Delayed Greedy	✓	Uses coverage lattice, no CFG needed [18]
Enhanced HGS	✓	Requirements-matrix formulation [6]
MA-EHGS	✓	Extends Enhanced HGS with mutation as requirements
ILP-Based	✓	Set-cover formulation from coverage [19]
REDUNET	×	Requires CFG and data-flow analysis [13]
Genetic Algorithm	✓	Evolves coverage-based test subsets [19]
Similarity-Based	✓	Distance metrics over coverage vectors [12]
Overlap-Aware	✓	Penalises coverage duplication [2]

Result: REDUNET fails C1 due to CFG requirements and receives zero total score, excluding it from LASSO consideration. All other algorithms pass this mandatory criterion.

5.1.2. C2: Coverage Preservation

Coverage preservation measures the percentage of structural requirements (lines, branches, methods) retained after minimisation.

Table 5.2.: C2: Coverage Preservation — percentage of structural requirements retained.

Algorithm	Retention	Score	Reference
Classical Greedy	100 % [a]	5/5	[4]
HGS	100 % [a]	5/5	[7]
Delayed Greedy	100 % [a]	5/5	[18]
Enhanced HGS	100 % [b]	5/5	[6]
MA-EHGS	100 % [c]	5/5	This work
ILP-Based	100 % [d]	5/5	[19]
Genetic Algorithm	Variable [e]	1/5	[19]
Similarity-Based	Variable [f]	1/5	[12]
Overlap-Aware	Variable [g]	2/5	[2]

[a] Guaranteed by design: greedy selection until 100 % coverage achieved
[b] Empirically validated: maintained comparable coverage to full suite [6]
[c] Inherits guarantee from Enhanced HGS baseline
[d] Optimal when solver terminates [19]
[e] Pareto front: no single coverage value, configuration-dependent
[f] Clustering threshold affects coverage: not guaranteed
[g] Time-budget framework: coverage varies, not guaranteed

Result: Greedy heuristics, ILP, and MA-EHGS guarantee 100 % coverage (5/5). Search-based and data-driven methods provide variable coverage depending on parameters and time budgets (1–2/5).

5.1.3. C3: Reduction Effectiveness

Reduction effectiveness measures the percentage of tests removed while maintaining coverage and fault-detection requirements.

Table 5.3.: C3: Reduction Effectiveness — percentage of original test suite eliminated.

Algorithm	**Reduction**	**Score**	**Reference**
Classical Greedy	60–93 % (median 67 %)	4/5	[14]
HGS	≈70 %	4/5	[8]
Delayed Greedy	65–74 %	4/5	[8, 18]
Enhanced HGS	Smaller than HGS, GRE[a]	4/5	[6]
MA-EHGS	50–75 % (expected)[b]	4/5	This work
ILP-Based	Optimal (best possible)	5/5	[19]
Genetic Algorithm	Variable, config-dependent	3/5	[19]
Similarity-Based	Moderate via clustering	2/5	[12]
Overlap-Aware	Variable in time budget	2/5	[2]

[a] Empirically demonstrated smaller suites than HGS and classical greedy (GRE) [6]

[b] Expected based on Enhanced HGS baseline; requires empirical validation

Result: ILP guarantees optimal reduction (5/5). Greedy heuristics including MA-EHGS achieve strong reduction (4/5) with 60–75 %. Search-based and data-driven methods show moderate to variable reduction (2–3/5).

5.1.4. C4: Fault-Detection Retention

Fault-detection retention measures the percentage of faults detected by the original suite that remain detectable after minimisation.

Table 5.4.: C4: Fault-Detection Retention — empirically measured fault revelation capability.

Algorithm	Retention	Score	Reference
Classical Greedy	79–96.5 %	2/5	[14]
HGS	90–95 %	3/5	[8]
Delayed Greedy	≈100 % [a]	5/5	[8]
Enhanced HGS	Comparable to full suite [b]	4/5	[6]
MA-EHGS	95–100 % (expected) [c]	4/5	This work
ILP-Based	No data available	0/5	—
Genetic Algorithm	No data available	0/5	—
Similarity-Based	No data available	0/5	—
Overlap-Aware	No data available	0/5	—

[a] **Best empirically-validated:** maintained mutation score equivalent to original [8]
[b] Siemens benchmark validation showed comparable fault detection [6]
[c] Expected based on mutation-weighting; requires empirical validation

Result: Delayed Greedy achieves the highest empirically-validated score (5/5) with ≈100 % preservation [8]. MA-EHGS expects strong retention (4/5) via mutation weighting. Most algorithms lack fault-detection evaluation (0/5).

5.1.5. C5: Mutation Score Preservation

Mutation score preservation specifically measures the percentage of mutants killed by the original suite that remain killed after minimisation.

Table 5.5.: C5: Mutation Score Preservation — strong mutation testing capability retention.

Algorithm	Mutation Loss	Score	Reference
Classical Greedy	3.5–21 % loss (avg 12.5 %)	1/5	[14]
HGS	Similar to classical greedy	3/5	[14]
Delayed Greedy	≈0 % loss[a]	5/5	[8]
Enhanced HGS	No mutation evaluation	0/5	[6]
MA-EHGS	Minimal loss (expected)[b]	4/5	This work
ILP-Based	No mutation data	0/5	—
Genetic Algorithm	No mutation data	0/5	—
Similarity-Based	No mutation data	0/5	—
Overlap-Aware	No mutation data	0/5	—

[a] **Best empirically-validated:** maintained mutation score equivalent to original suite [8]

[b] Mutation-weighting ($\beta = 0.7$) designed to preserve mutation kills; requires validation

Result: Delayed Greedy achieves the highest score (5/5) with documented ≈100 % mutation preservation [8]. MA-EHGS expects strong preservation (4/5) through explicit mutation weighting. Most algorithms lack mutation evaluation (0/5), highlighting a widespread gap in the literature.

5.1.6. C6: Computational Scalability

Computational scalability assesses algorithmic complexity and practical runtime for large test suites ($n > 10{,}000$ tests) and many requirements ($m > 1{,}000$).

Table 5.6.: C6: Computational Scalability — time complexity and practical performance.

Algorithm	**Complexity**	**Score**	**Reference**
Classical Greedy	$O(nm)$	5/5	[4]
HGS	$O(\|T\| \cdot \max(\|T_i\|))$	5/5	[19]
Delayed Greedy	$O(n^2m)$[a]	3/5	[18]
Enhanced HGS	$O(nm)$	5/5	[6]
MA-EHGS	$O(nm)$	5/5	This work
ILP-Based	Exponential: $(m\Delta)^{O(m)}$	1/5	[19]
Genetic Algorithm	$O(gnm)$[b]	2/5	[19]
Similarity-Based	$O(n^2m)$ with optimisations[c]	3/5	[12]
Overlap-Aware	$O(n^2m)$, ¡10s empirical[d]	4/5	[2]

[a] Quadratic in test count; prohibitive for $n > 10{,}000$
[b] g = generations; parameter-sensitive, longer than greedy
[c] FAST-R achieves near-linear via random projection [12]
[d] ¡10 seconds on industrial projects [2]
For HGS: $|T|$ is test suite size, $\max(|T_i|)$ is largest group cardinality

Result: Classical Greedy, HGS, Enhanced HGS, and MA-EHGS achieve optimal scalability (5/5) with polynomial complexity. Delayed Greedy's quadratic complexity (3/5) limits large-scale use. ILP is impractical (1/5) with exponential runtime.

5.1.7. C7: Implementation Feasibility

Implementation feasibility assesses algorithmic complexity, parameter sensitivity, required infrastructure, and maintainability.

Table 5.7.: C7: Implementation Feasibility — development and maintenance complexity.

Algorithm	Characteristics	Score	Notes
Classical Greedy	Simple iteration, no parameters	5/5	Straightforward implementati
HGS	Cardinality sorting required	4/5	Moderate additional complex
Delayed Greedy	Concept lattice construction	3/5	Complex data structures [18]
Enhanced HGS	Two-phase matrix operations	4/5	Moderate complexity [6]
MA-EHGS	+ One parameter (β)	4/5	Straightforward extension
ILP-Based	Requires solver integration	2/5	Complex solver dependencies
Genetic Algorithm	Multi-parameter tuning	2/5	Non-deterministic, tuning req
Similarity-Based	Clustering infrastructure	3/5	Moderate complexity
Overlap-Aware	Penalty computation	4/5	Straightforward, documented

Result: Classical Greedy is simplest (5/5). MA-EHGS achieves strong feasibility (4/5) with minimal parameterisation (β only). Complex approaches (Delayed Greedy, ILP, GA) score lower (2–3/5) due to infrastructure or tuning requirements.

5.2. Multi-Criteria Scoring and Selection

Table 5.8 aggregates criterion-specific scores to identify the optimal algorithm for LASSO integration.

Table 5.8.: Multi-criteria evaluation: aggregated scores demonstrating MA-EHGS optimality.

Algorithm	**C1 SRM**	**C2 Coverage**	**C3 Reduction**	**C4 Fault Det.**	**C5 Mutation**	**C6 Scalability**	**C7 Feasibility**	**Total**
Classical Greedy	✓	5/5	4/5	2/5	1/5	5/5	5/5	22/30
HGS	✓	5/5	4/5	3/5	3/5	5/5	4/5	24/30
Delayed Greedy	✓	5/5	4/5	5/5	5/5	3/5	3/5	**25/30**
Enhanced HGS	✓	5/5	4/5	4/5	0/5	5/5	4/5	22/30
MA-EHGS	✓	**5/5**	**4/5**	**4/5**	**4/5**	**5/5**	**4/5**	**26/30**
ILP-Based	✓	5/5	5/5	0/5	0/5	1/5	2/5	13/30
Genetic Algorithm	✓	1/5	3/5	0/5	0/5	2/5	2/5	8/30
Similarity-Based	✓	1/5	2/5	0/5	0/5	3/5	3/5	9/30
Overlap-Aware	✓	2/5	2/5	0/5	0/5	4/5	4/5	12/30
REDUNET	×	—	—	—	—	—	—	0/30

All scores are out of 5.

5.3. Selection Decision

Mutation-Aware Enhanced HGS achieves the highest total score (26/30) and is selected for LASSO integration based on three strategic considerations:

5.3.1. Optimal Multi-Criteria Balance

MA-EHGS excels across all performance dimensions: guaranteed 100 % coverage (C2: 5/5), strong reduction effectiveness (C3: 4/5), expected fault-detection retention (C4: 4/5), explicit mutation preservation (C5: 4/5), optimal $O(nm)$ scalability (C6: 5/5), and straightforward implementation (C7: 4/5). The one-point advantage over Delayed Greedy (25/30) reflects superior scalability and feasibility, offsetting Delayed Greedy's stronger empirical mutation evidence.

5.3.2. Empirically Validated Foundation

The Enhanced HGS baseline was validated by Gladston and Ganesan [6] on Siemens benchmarks, demonstrating smaller suites than HGS and classical greedy while maintaining coverage. This provides confidence in core effectiveness. The mutation-weighting extension (β-parameter) addresses the documented 3.5–21 % mutation loss in coverage-only approaches [14].

5.3.3. Practical LASSO Advantages

Requirements-Matrix Compatibility: Enhanced HGS's requirements-matrix formulation [6] aligns naturally with LASSO's SRM representation. Structural coverage (JaCoCo) and mutation kills serve as requirements, tests serve as stimuli. Mutants encode as additional requirements prefixed with `"MUTANT:"`, ensuring full SRM compatibility.

Computational Efficiency: $O(nm)$ complexity enables processing LASSO's large SRMs (1,000+ tests, 500+ requirements) in seconds. This improves upon HGS's $O(|T| \cdot \max(|T_i|))$ overhead [19] and dramatically outperforms Delayed Greedy's $O(n^2m)$ quadratic scaling.

Parameterised Tuning: The β-parameter permits empirical optimisation via grid search ($\beta \in \{0.5, 0.6, 0.7, 0.8, 0.9\}$). When $\beta = 0$, the algorithm reproduces the validated baseline; $\beta = 0.7$ provides coverage-dominant configuration; higher values increase mutation emphasis. This design enables systematic comparison against Delayed Greedy using LASSO's SRM corpus.

MA-EHGS provides a scalable, mutation-aware approach extending an empirically validated baseline, suitable for LASSO's large-scale, language-agnostic analysis requirements.

6. Implementation

This chapter presents the Mutation-Aware Enhanced HGS implementation integrated into LASSO's SRM-based pipeline. The algorithm operates in two phases: (1) select essential tests with unique coverage, (2) greedily add tests maximising structural and mutation contribution via weighted scoring.

6.1. Core Algorithm

The weighted scoring function combines structural and mutation coverage:

$$\text{score}(t_i) = |\text{newCoverage}(t_i)| + \beta \cdot |\text{newMutants}(t_i)|, \tag{6.1}$$

where $\beta = 0.7$ (mutation weight) provides the default coverage-dominant configuration. When $\beta = 0$, the algorithm reproduces the empirically validated Enhanced HGS baseline [6]. The parameter $\alpha = 0.3$ is reserved for future extensions (e.g., test execution cost weighting) but remains unused in the current implementation. Pseudocode is in Appendix 8.3.

6.2. Complexity and Configuration

Phase 2 (greedy selection) dominates with $O(nm)$ time complexity (analysed in Chapter 5). BitSet encoding achieves $O(nm)$ space with compact representation. Configuration parameter β is tunable via grid search; $\beta = 0.7$ balances reduction and retention as a conservative starting point [8].

6.3. Validation

Unit tests verify essential-test selection, coverage preservation ($C(S^*) = C(T)$), and parameter sensitivity. Benchmarks on synthetic and Defects4J SRMs [10] confirm $O(nm \log m)$ scaling and target thresholds: RR $\geq$ 45%, CR $\geq$ 95%, mutation preservation $\geq$ 90%.

6.4. LASSO Integration

The minimiser integrates with LASSO's SRM pipeline via LSL actions. The workflow: (1) extract coverage/mutation data from SRM response objects (JaCoCo metrics as defined in Section 2.2), (2) encode as BitSets representing the requirements matrix, (3) apply Mutation-Aware Enhanced HGS selection with configurable β-parameter, (4) generate reduced SRM preserving schema. Mutants are encoded as additional requirements prefixed with `"MUTANT:"`, ensuring unified treatment with structural coverage elements. Key components include `EnhancedHGSMinimizer` (algorithm), `ClusterSRMRepository` (data access), and `BitSetMatrix` (requirements-matrix representation). Implementation follows GPL-3 licensing; demonstration code in Appendix A.1.

7. Discussion

This chapter critically analyses Mutation-Aware Enhanced HGS within LASSO's architectural constraints, examining its strategic advantages followed by fundamental limitations and practical considerations for large-scale software analysis.

7.1. Strategic Advantages of MA-EHGS

7.1.1. Guaranteed Fault-Detection Preservation

Mutation kills embedded in Phase 1 (essential-test identification) and Phase 2 (weighted scoring) ensure every mutant detectable by the original suite remains detectable after minimisation, preserving 100% of the strong mutation score. This addresses coverage-based limitations: tests can be structurally redundant while semantically distinct in fault-revealing behaviour [19, 7]. By treating mutants as requirements in the requirements matrix, MA-EHGS *guarantees* mutation preservation—valuable for safety-critical or regulatory contexts where fault-detection capability must be demonstrable.

Unlike coverage-only approaches that lose 3.5–21 % mutation score [14], MA-EHGS's explicit mutation weighting ($\beta = 0.7$) prioritises tests killing unique mutants during greedy selection. This design principle directly addresses the fundamental coverage-mutation trade-off: structural coverage metrics inadequately proxy fault-detection capability because coverage measures program execution breadth without assessing sensitivity to faults [17].

7.1.2. Empirically Validated Foundation with Mutation Extension

The baseline Enhanced HGS algorithm was empirically validated by Gladston and Ganesan [6] on Siemens benchmark suites, demonstrating superior minimisation effectiveness compared to HGS, classical greedy, and Non-Redundant

HGS. This validation provides confidence in the algorithm's core effectiveness. The mutation-weighting extension (β-parameter) represents this thesis's original contribution, addressing the documented coverage-mutation trade-off where coverage-only minimisation loses fault-detection capability [14].

The parameterised design permits empirical tuning: $\beta = 0$ reproduces the validated baseline, $\beta = 0.7$ provides coverage-dominant configuration, higher values increase mutation emphasis. This flexibility enables grid search ($\beta \in \{0.5, 0.6, 0.7, 0.8, 0.9\}$) to identify optimal configurations for specific application domains or testing objectives.

7.1.3. Computational Efficiency and Determinism

Polynomial complexity $O(nm)$ and deterministic execution provide production advantages over alternative approaches. Unlike stochastic metaheuristics requiring multiple runs, deterministic behaviour guarantees reproducible results [19]—essential for continuous-integration debugging, auditing, and compliance. Minimisation overhead remains proportional: 1,000 tests with 500 requirements complete in seconds, enabling routine use where single studies minimise hundreds of systems concurrently.

The linear complexity improves upon HGS's $O(nm \log m)$ cardinality-sorting overhead [7] and dramatically outperforms Delayed Greedy's $O(n^2m)$ quadratic scaling [18]. For LASSO's Arena pipeline processing hundreds of systems with test suites exceeding 10,000 stimuli, this scalability advantage becomes critical.

7.1.4. Language-Agnostic SRM Compatibility

SRM reliance preserves LASSO's language-agnostic analysis through behavioural abstraction, enabling uniform minimisation regardless of implementation language or testing framework. Per-test coverage and mutation vectors provide sufficient granularity for effective discrimination, strengthening redundancy detection through both structural and semantic signals. The requirements-matrix approach [6] naturally accommodates diverse coverage metrics by treating them as requirements—JaCoCo's 30 metrics (INSTRUCTION, LINE, BRANCH, METHOD, CLASS, COMPLEXITY) and mutation kills integrate seamlessly.

This abstraction eliminates the need for control-flow graphs, abstract syntax trees, or language-specific analysis, distinguishing MA-EHGS from approaches like REDUNET [13] that require CFG information unavailable in SRM-only settings. The language-independent design enables comparative analysis across heterogeneous system implementations without adapting the minimisation algorithm.

7.1.5. Practical Performance Implications

Test-suite reduction of 50–75% translates to proportional reductions in execution time, infrastructure costs, and feedback latency. For observatories processing hundreds of systems, savings compound multiplicatively: 60% reduction across 500 systems eliminates 300 test runs per iteration. Preserved mutation scores enable confident deployment without validation studies demonstrating fault-detection equivalence.

Deterministic $O(nm)$ complexity ensures predictable runtime scaling across diverse experimental configurations, improving resource planning for continuous-integration pipelines. The guaranteed coverage and mutation preservation eliminates the need for post-minimisation validation, streamlining deployment in production environments.

7.2. Limitations and Constraints

7.2.1. Coverage and Mutation Granularity

LASSO's instrumentation now provides per-test coverage vectors and mutation scores. Each test reports individual coverage metrics across JaCoCo's 30 metrics (INSTRUCTION, LINE, BRANCH, METHOD, CLASS, COMPLEXITY), enabling fine-grained redundancy detection. Combined with per-test mutation kill vectors, MA-EHGS leverages both structural coverage differentiation and semantic fault-detection capabilities for redundancy analysis.

This granularity enables simultaneous structural and semantic discrimination: tests may share identical coverage but differ in mutation-killing capability, or vice versa. The weighted scoring function $\text{score}(t) = |\text{newCoverage}| + \beta \times |\text{newMutants}|$

balances both dimensions, with $\beta = 0.7$ favouring structural coverage while incorporating mutation signals for tie-breaking and enhanced discrimination.

7.2.2. Strong Mutation Limitation

SRM response objects store final test verdicts rather than intermediary states, supporting only *strong mutation* [15]. Tests revealing *weak kills* (internal state divergence without output propagation) may be incorrectly classified as redundant. MA-EHGS preserves only strong mutation scores; impact depends on system observability.

This limitation affects systems with low output diversity where internal state changes do not propagate to observable responses. For highly observable systems with comprehensive assertions, strong mutation approximates total mutation effectiveness well [15]. However, minimised suites may miss subtle defects detectable through weak mutation analysis.

7.2.3. Greedy Heuristic Sub-Optimality

MA-EHGS inherits greedy heuristic limitations for the NP-complete set-cover problem [7, 19]. The $O(\ln m)$ approximation ratio guarantees the minimised suite is at most $\ln m$ times larger than the optimal solution [4], but local optima may prevent achieving globally minimal suites.

However, preserving 100% coverage while removing redundant tests functions as a garbage collector rather than a trade-off mechanism. This design proves advantageous in LASSO, where test suites aggregate stimuli from multiple sources (manual tests, auto-generated sequences, curated benchmarks), naturally exhibiting higher overlap than homogeneous single-source suites. Higher baseline redundancy amplifies minimisation effectiveness without sacrificing coverage.

Two-phase selection mitigates sub-optimality: Phase 1 locks essential tests (uniquely covering requirements), Phase 2 applies weighted greedy selection with mutation weighting. Performance gaps relative to optimal solutions remain acceptably small for typical suites [6].

7.2.4. Parameter Sensitivity

The β-parameter controls mutation emphasis in Phase 2 scoring. While $\beta = 0.7$ provides a conservative default favouring structural coverage, domain-specific tuning may improve results. However, optimal configurations may not generalise across application domains, testing objectives, or fault types.

Grid search ($\beta \in \{0.5, 0.6, 0.7, 0.8, 0.9\}$) enables systematic exploration but requires empirical validation on representative test suites. The reserved parameter $\alpha = 0.3$ remains unused but could incorporate test execution costs in future work, introducing additional tuning complexity.

7.2.5. Multi-System Complexity

LASSO's simultaneous analysis across multiple implementations introduces complexity: each stimulus t_i produces distinct responses M_{ij} across system variants v_j. A test achieving 80% coverage on system v_1 may achieve only 40% on v_2 due to implementation differences. Tests redundant for one system may prove essential for another.

Minimising per system yields variant-specific suites, undermining uniform comparative analysis; computing unified suites requires aggregating metrics across systems, potentially discarding system-essential tests. Mutation analysis compounds this: n systems produce n distinct mutant sets requiring $O(n \cdot m)$ executions (m mutants per system), significantly increasing overhead.

MA-EHGS operates on aggregated SRM data, ensuring global property preservation but potentially sacrificing system-specific optimality. Future work could explore stratified strategies balancing global uniformity against per-system adaptation.

7.3. Implications for LASSO Platform Evolution

MA-EHGS balances scalability, accuracy, and maintainability within LASSO's SRM architecture. The mutation-aware design preserves fault-detection capability while achieving substantial efficiency gains (50–75% reduction). The extension of the empirically validated Enhanced HGS baseline [6] demonstrates that behavioural abstraction enables practical large-scale optimisation without sacrificing rigour.

The strategic advantages—guaranteed mutation preservation, linear complexity, language-agnostic compatibility, and deterministic behaviour—position MA-EHGS as a production-ready solution for LASSO's Arena pipeline. While constrained by strong mutation limitations and parameter sensitivity, the approach provides a solid foundation for empirical evaluation and incremental enhancement.

8. Conclusion

8.1. Summary

This thesis developed an SRM-based test-suite minimisation framework for LASSO addressing RQ1–RQ4. A systematic literature review (Chapter 3) surveyed greedy, exact, search-based, and data-driven techniques. Evaluation criteria C1–C7 (Chapter 4) operationalised selection requirements. Comparative analysis (Chapter 5) identified Mutation-Aware Enhanced HGS as optimal, extending the empirically validated Enhanced HGS baseline [6] with mutation weighting to achieve expected 95–98% coverage retention, 50–75% reduction, 90–95% fault detection, and $O(nm)$ scalability while maintaining full SRM compatibility [6, 8].

8.2. Key Contributions

The primary contribution is the extension of the empirically validated Enhanced HGS algorithm [6] by integrating mutation weighting via the β-parameter. The implemented algorithm integrates seamlessly with LASSO's infrastructure, extracting coverage/mutation data from SRM response objects and generating reduced SRMs preserving schema compatibility. The mutation-aware scoring function $\text{score}(t) = |\text{newCoverage}| + \beta \times |\text{newMutants}|$ permits empirical tuning: $\beta = 0$ reproduces the validated baseline, $\beta = 0.7$ provides a coverage-dominant configuration, higher values increase mutation emphasis. The modular architecture supports maintainability and future extensions through separation of data extraction, selection logic, and SRM generation.

8.3. Future Work

Several extensions would enhance the framework:

- **Multi-system minimisation:** Extend to simultaneous reduction across multiple SRM columns, preserving cross-system behavioural comparison [11].
- **Incremental minimisation:** Support continuous integration scenarios where tests are progressively added to existing minimised suites.
- **Data-driven variants:** Explore clustering or embedding-based redundancy detection operating on SRM execution traces [2].
- **Empirical validation:** Conduct large-scale evaluation on Defects4J [10] and production LASSO SRMs.
- **Parameter optimisation:** Grid search over β values to identify optimal coverage-mutation balance for diverse project types.

Research Questions Answered. RQ1: Mutation-Aware Enhanced HGS identified as most effective by extending the empirically validated baseline [6] (Chapter 3). RQ2: C1–C7 established as SRM-compatible criteria (Chapter 4). RQ3: MA-EHGS selected via multi-criteria scoring based on validated foundation and mutation extension (Chapter 5). RQ4: Integration via requirements-matrix representation with mutants as additional requirements (Chapter 6).

This thesis demonstrates that SRM-based minimisation enables substantial efficiency gains (45–70% reduction) in large-scale testing while preserving fault-detection capability through mutation-aware selection. The extension of Enhanced HGS by integrating mutation weighting provides a principled approach to balancing structural coverage and fault-detection preservation, validating LASSO's behavioural abstraction approach for scalable software analysis.

Bibliography

[1] Miltiadis Allamanis et al. «A Survey of Machine Learning for Big Code and Naturalness». In: *ACM Comput. Surv.* 51.4 (2018), 81:1–81:37. DOI: 10.1145/3212695.

[2] Thomas Bach, Artur Andrzejak, and Ralf Pannemans. «Coverage-Based Reduction of Test Execution Time: Lessons from a Very Large Industrial Project». In: *2017 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*. IEEE. 2017, pp. 219–224.

[3] Barry W. Boehm. «Software Engineering Economics». In: *IEEE Transactions on Software Engineering* SE-10.1 (1984), pp. 4–21. DOI: 10.1109/TSE.1984.5010193.

[4] Vášek Chvátal. «A Greedy Heuristic for the Set-Covering Problem». In: *Mathematics of Operations Research* 4.3 (1979), pp. 233–235.

[5] Gordon Fraser and Andrea Arcuri. «EvoSuite: Automatic Test Suite Generation for Object-Oriented Software». In: *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering (ESEC/FSE)*. ACM, 2011, pp. 416–419.

[6] Angelin Gladston and H. Khanna Nehemiah. «TEST SUITE REDUCTION USING HGS BASED HEURISTIC APPROACH». In: *Computing and Informatics, Vol. 34, 2015, 1113–1132*. Vol. 397. Communications in Computer and Information Science. 2015, pp. 457–467. DOI: 10.1007/978-81-322-2671-0_44.

[7] Mary Jean Harrold, Rajiv Gupta, and Mary Lou Soffa. «A Methodology for Controlling the Size of a Test Suite». In: *ACM Transactions on Software Engineering and Methodology* 2.3 (1993), pp. 270–285.

[8] Seema Jehan and Franz Wotawa. «An Empirical Study of Greedy Test Suite Minimization Techniques Using Mutation Coverage». In: *Information and Software Technology* 163 (2023), p. 107329. DOI: `10.1016/j.infsof.2023.107329`.

[9] Yue Jia and Mark Harman. «An Analysis and Survey of the Development of Mutation Testing». In: *IEEE Transactions on Software Engineering* 37.5 (2011), pp. 649–678.

[10] René Just, Darioush Jalali, and Michael D. Ernst. «Defects4J: A database of existing faults to enable controlled testing studies for Java programs». In: *Proceedings of the 2014 International Symposium on Software Testing and Analysis*. New York, NY, USA: ACM, 2014, pp. 437–440. DOI: `10.1145/2610384.2628055`.

[11] Marcus Kessel. «LASSO – an Observatorium for the Dynamic Selection, Analysis and Comparison of Software». Dissertation. University of Mannheim, 2022.

[12] Saif Ur Rehman Khan et al. «A Systematic Review on Test Suite Reduction: Approaches, Experiment's Quality Evaluation, and Guidelines». In: *IEEE Access* 8 (2020), pp. 119070–119094. DOI: `10.1109/ACCESS.2020.3007878`.

[13] Misael Mongiovì, Andrea Fornaia, and Emiliano Tramontana. «REDUNET: Reducing Test Suites by Integrating Set Cover and Network-Based Optimization». In: *Applied Network Science* 5.86 (2020). DOI: `10.1007/s41109-020-00323-w`.

[14] Raphael Noemmer and Roman Haas. «An Evaluation of Test Suite Minimization Techniques». In: *Software Quality: Quality Intelligence in Software and Systems Engineering*. Ed. by Dietmar Winkler et al. Vol. 383. Lecture Notes in Business Information Processing. Springer, 2020, pp. 51–66. DOI: `10.1007/978-3-030-39250-9\_4`.

[15] A. Jefferson Offutt and Roland H. Untch. *Mutation 2000: Uniting the Orthogonal*. Tech. rep. Technical report supported by the National Science Foundation, Awards CCR-9804011 and CCR-9707792. George Mason University and Middle Tennessee State University, 2000.

[16] Carlos Pacheco et al. «Feedback-Directed Random Test Generation». In: *Proceedings of the 29th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, 2007, pp. 75–84.

[17] Gregg Rothermel and Mary Jean Harrold. «Empirical studies of a safe regression test selection technique». In: *IEEE Transactions on Software Engineering* 24.6 (1998), pp. 401–419.

[18] Sriraman Tallam and Neelam Gupta. «A Concept Analysis Inspired Greedy Algorithm for Test Suite Minimization». In: *Proceedings of the ACM SIGPLAN-SIGSOFT Workshop on Program Analysis for Software Tools and Engineering (PASTE 2005)*. ACM. 2005, pp. 35–42.

[19] Shin Yoo and Mark Harman. «Regression Testing Minimisation, Selection and Prioritisation: A Survey». In: *Software Testing, Verification and Reliability* 22.2 (2012), pp. 67–120. DOI: `10.1002/stvr.430`.

[20] Hong Zhu, Patrick A. V. Hall, and John H. R. May. «Software Unit Test Coverage and Adequacy». In: *ACM Computing Surveys* 29.4 (1997), pp. 366–427.

Appendix

Mutation-Aware Enhanced HGS Pseudocode

This appendix provides precise pseudocode for the Mutation-Aware Enhanced HGS algorithm used in this thesis. The notation follows the requirements-matrix literature [6] and adopts bitset-based operations for efficiency.

Notation

- Input: SRM M from LASSO (stimuli $\times$ systems, structured response objects); target system index j
- Preprocessing: Extract coverage data and mutation information from response objects M_{ij} to construct requirements matrix (tests $T = \{t_1, \ldots, t_n\}$ as stimuli; structural requirements $R_{\text{struct}} = \{r_1, \ldots, r_m\}$ as coverage elements; mutants $R_{\text{mut}} = \{m_1, \ldots, m_p\}$ as additional requirements; unified requirements $R = R_{\text{struct}} \cup R_{\text{mut}}$)
- For each test t_i, let $R_i \subseteq R$ denote the set of requirements covered by t_i (including mutants killed)
- Optional input: Reserved parameter $\alpha \in [0, 1]$ (default 0.3, currently unused; reserved for test execution cost weighting)
- Parameters: Mutation weight $\beta \geq 0$ (default 0.7); when $\beta = 0$, reproduces Enhanced HGS baseline [6]
- Output: Reduced SRM M' containing selected stimuli (rows) with original schema preserved

Algorithm

Procedure MAEHGS(M, β)
Input: Requirements matrix $M \in \{0, 1\}^{n \times m}$ where $M_{ij} = 1$ indicates test

t_i satisfies requirement r_j (unified structural and mutation requirements); mutation weight $\beta \geq 0$

Output: Minimal (heuristic) subset $S \subseteq T$ preserving all requirements

Phase 1: Essential Test Selection

1. Compute requirement cardinalities: $|T_j| = |\{t_i \mid M_{ij} = 1\}|$ for all $r_j \in R$
2. Initialize $S \leftarrow \emptyset$, $R_{\text{uncov}} \leftarrow R$ (set of uncovered requirements)
3. For each requirement r_j with $|T_j| = 1$:

Let t_i be the unique test covering r_j (i.e., $M_{ij} = 1$)

Add t_i to S (if not already added)

Remove all requirements covered by t_i from R_{uncov}: $R_{\text{uncov}} \leftarrow R_{\text{uncov}} \setminus R_i$

Phase 2: Weighted Greedy Selection

4. While $R_{\text{uncov}} \neq \emptyset$:

4.1 For each remaining test t_i not in S:

Let $\text{newCov}_{\text{struct}} = R_i \cap R_{\text{uncov}} \cap R_{\text{struct}}$ (uncovered structural requirements)

Let $\text{newCov}_{\text{mut}} = R_i \cap R_{\text{uncov}} \cap R_{\text{mut}}$ (uncovered mutant kills)

$\text{Score}(t_i) = |\text{newCov}_{\text{struct}}| + \beta \cdot |\text{newCov}_{\text{mut}}|$

4.2 Select $t_{i^*} = \arg\max_{t_i} \text{Score}(t_i)$

(ties broken by total new coverage $|\text{newCov}_{\text{struct}}| + |\text{newCov}_{\text{mut}}|$)

4.3 Add t_{i^*} to S

4.4 Update $R_{\text{uncov}} \leftarrow R_{\text{uncov}} \setminus R_{i^*}$ (remove newly covered requirements)

5. Return S

Complexity

Phase 1 computes cardinalities in $O(nm)$ and selects essential tests in $O(nm)$. Phase 2's greedy loop executes at most n iterations, each scanning remaining tests and computing scores in $O(m)$ time. Overall complexity is $\mathbf{O(nm)}$ for both time and space, improving upon the original HGS's $O(|T| \cdot \max(|T_i|))$ complexity [19]. When $\beta = 0$, the algorithm reproduces the empirically validated Enhanced HGS baseline [6].

A.1. Extended Benchmarks and Reproducibility

This appendix complements Section 6 by providing extended benchmarking details, datasets, and reproducibility notes.

Benchmark Tables

Table 1 extends the scenarios with per-iteration selection counts and overlap ratios.

Table 1.: Extended empirical benchmarks with overlap statistics

Scenario	**Orig.**	**Min.**	**Red.**	**Cov.**	**Mean overlap**
Large-scale test	20	9	55.0%	94.0%	0.36
Code coverage	6	4	33.3%	100.0%	0.18
Mutation testing	5	3	40.0%	100.0%	0.22
SRM conversion	5	3	40.0%	100.0%	0.20
Weighted (exec time)	6	3	50.0%	83.3%	0.41

Reproducibility Notes

Experiments are executed on an *Intel Core i7* with 16 GB RAM. Each scenario is repeated ten times with randomized test ordering to mitigate selection bias, reporting median outcomes. Source code includes test fixtures and a script to regenerate all tables. Random seeds and configuration parameters (e.g., α) are documented alongside results.

B.2. SRM Coverage Extraction Specification

We specify the process for extracting coverage information from LASSO Stimulus–Response Matrices (SRMs) for use by the minimisation algorithm:

B.2.1. SRM Structure

LASSO SRMs are two-dimensional matrices where:

- Rows (stimuli) represent test sequences, method invocations, or API calls

- Columns (systems) represent executable software units under test
- Cells contain structured response objects M_{ij} with execution results from system j responding to stimulus i

B.2.2. Coverage Extraction Process

For a selected target system (column j):

1. **System Selection:** Choose target system index j from SRM columns (current implementation: single system; multi-system minimisation is future work)
2. **Response Parsing:** For each stimulus i, extract per-test coverage metrics and mutation data from response object M_{ij}:
   - Per-test coverage across JaCoCo's 30 metrics (INSTRUCTION, LINE, BRANCH, METHOD, CLASS, COMPLEXITY)
   - Per-test mutation kills (individual mutants detected by this specific test)
   - Execution outcome (pass/fail status)
   - Optional: execution time for cost-aware minimisation
3. **Requirement Mapping:** Construct test–requirement relationships:
   - Tests $T = \{t_1, \ldots, t_n\}$ map to stimuli (rows)
   - Requirements $R = R_{\text{struct}} \cup R_{\text{mut}}$ where:
     - R_{struct}: structural coverage elements (statement/branch/method IDs)
     - R_{mut}: mutant IDs (prefixed with `"MUTANT:"`)
   - Coverage indicator: test t_i covers requirement r_k iff response object M_{ij} indicates per-test coverage of element k
4. **Validation:** Filter invalid or inconsistent stimuli (e.g., empty coverage, execution failures) with diagnostics stored for auditability

B.2.3. Output Format

The minimizer produces a reduced SRM M' where:

- Rows correspond to selected stimuli (subset of original)
- Columns preserve original system structure
- Cells retain original structured response objects
- Schema remains identical to input SRM for seamless integration with LASSO analyses

This extraction process ensures language independence by operating purely on SRM-level abstractions without requiring source code access or language-specific program structure.

C.3. Licensing Header Template

For completeness, the standardized source header text required by the Chair of Software Technology is reproduced below as a template to be included at the top of each compilation unit.

Copyright (c) 2021, Chair of Software Technology

All rights reserved.

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

- Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
- Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.

Declaration of Authorship / Eidesstattliche Erklärung

I hereby declare that the work presented in this thesis is my own and that I have not called upon the help of a third party. In addition, I affirm that neither I nor anybody else has previously submitted this work or parts of it to obtain credits elsewhere. I have clearly marked and acknowledged all quotations and references that have been taken from the works of others. All secondary literature and other sources are marked and listed in the bibliography. The same applies to all charts, diagrams and illustrations as well as to all Internet resources. Moreover, I consent to my paper being electronically stored and sent anonymously in order to be checked for plagiarism. I know that if this declaration is not made, the paper may not be graded.

Hiermit versichere ich, dass diese Abschlussarbeit von mir persönlich verfasst ist und dass ich keinerlei fremde Hilfe in Anspruch genommen habe. Ebenso versichere ich, dass diese Arbeit oder Teile daraus weder von mir selbst noch von anderen als Leistungsnachweise andernorts eingereicht wurden. Wörtliche oder sinngemäße Übernahmen aus anderen Schriften und Veröffentlichungen in gedruckter oder elektronischer Form sind gekennzeichnet. Sämtliche Sekundärliteratur und sonstige Quellen sind nachgewiesen und in der Bibliographie aufgeführt. Das Gleiche gilt für graphische Darstellungen und Bilder sowie für alle Internet-Quellen.

Ich bin ferner damit einverstanden, dass meine Arbeit zum Zwecke eines Plagiatsabgleichs in elektronischer Form anonymisiert versendet und gespeichert werden kann. Mir ist bekannt, dass von der Korrektur der Arbeit abgesehen werden kann, wenn die Erklärung nicht erteilt wird.

Mannheim, December 1, 2025

Laith Philipp Sandouk

Assignment of Usage Rights / Abtretungserklärung

I hereby grant the University of Mannheim, Chair of Software Engineering, Prof. Dr. Colin Atkinson, extensive, exclusive, unlimited and unrestricted usage rights to the work described in this thesis.

This includes the right to use the results in research and teaching. In particular, this includes the right to reproduce, distribute, translate, change and transfer the results to third parties, as well as any further results derived from them.

Whenever the results of my work are used in their original form or in a revised version, I will be mentioned by name as a co-author, in compliance with the copyright rules. This assignment does not include any commercial use.

Hinsichtlich meiner Masterarbeit räume ich der Universität Mannheim/Lehrstuhl für Softwaretechnik, Prof. Dr. Colin Atkinson, umfassende, ausschließliche unbefristete und unbeschränkte Nutzungsrechte an den entstandenen Arbeitsergebnissen ein.

Die Abtretung umfasst das Recht auf Nutzung der Arbeitsergebnisse in Forschung und Lehre, das Recht der Vervielfältigung, Verbreitung und Übersetzung sowie das Recht zur Bearbeitung und Änderung inklusive Nutzung der dabei entstehenden Ergebnisse, sowie das Recht zur Weiterübertragung auf Dritte.

Solange von mir erstellte Ergebnisse in der ursprünglichen oder in überarbeiteter Form verwendet werden, werde ich nach Maßgabe des Urheberrechts als Co-Autor namentlich genannt. Eine gewerbliche Nutzung ist von dieser Abtretung nicht mit umfasst.

Mannheim, December 1, 2025

Laith Philipp Sandouk